

Práctica 1: Configuración del Entorno

Análisis de Rendimiento de los lenguajes de Programación

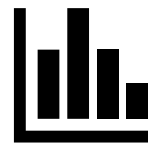
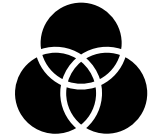
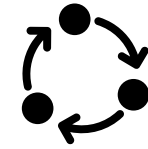
AED I — Bloque I

2 h en clase + 1 h en casa

Algoritmia y Estructuras de Datos | Ingeniería Informática – Inteligencia Artificial

¿Qué vamos a hacer?

1. Configurar entorno de ejecución en Python, C++ y Java
2. Ver diferencias básicas de sintaxis con programas básicas
3. Medir tiempos de ejecución y comparar rendimiento (benchmark)
4. Usar entorno de programación competitiva para evaluación



Objetivos de la práctica

1. Entender sintaxis básica de los 3 lenguajes con ejercicios en vivo
2. Medir tiempos de ejecución reales y entender trade-offs
3. Establecer prácticas de programación competitiva

Flujo de la sesión

Fase	Duración	Actividad
Instalación y verificación de entorno	15 min	Verificar versión
Ejercicio 1: Hola Mundo	10 min	Escribir, compilar y ejecutar en los 3 lenguajes
Ejercicio 2: Implementar Factorial	20 min	Implementar
Ejercicio 3: Comparativa de rendimiento	30 min	Implementar sumar de matriz, medir tiempos y discutir resultados
Ejercicio 4: Competir con nuestro programa	25 min	Preparar programa para juez virtual
Cierre y Reflexión	10 min	Tabla comparativa, elección de lenguaje para competición

Lenguajes y entornos de desarrollo

Lenguaje	Versión mínima	Compilador	Entorno (IDE)
Python	3.8+	Compilador Python + pip	PyCharm Community/Jupyter Notebook/ VSCode con extensión Python
C++	C++17+	G++ (MinGW/xcode)	Code::Blocks/Clion (Jetbrains)/ VSCode con extensión C++
Java	Java SE 11+ (LTS)	OpenJDK	JDK+ IntelliJ IDEA Community/Eclipse/ VS Code con extensión Java
Terminal / Bash	—		Compilación y ejecución

Setup

Sistema Operativo	Comando instalador (en terminal)	Python	C++	Java
Windows	choco install	python	Mingw	openjdk11
macOS	brew install	python3	(xcode-select --install) gcc	openjdk@11
Linux	(sudo apt update) sudo apt install	python3 python3-pip	build-essential	openjdk-11-jdk

Verifica que todo funciona antes de empezar:

```
$ python --version # → Python 3.8+  
$ java -version    # → OpenJDK 11+  
$ javac -version   # → javac 11+  
$ g++ --version    # → g++ 9+ (C++14+)
```

Tip: Crea una carpeta por práctica: practica1/ con subcarpetas python/, java/, cpp/

Problema común en Windows: g++ no está en PATH. Solución: Usar MinGW-w64 o WSL2.

Ejercicio 1: Hola Mundo

```
# Python – hola.py (no necesita compilación)
print("¡Hola Mundo desde Python!")
print(f"Versión: {__import__('sys').version}")
// python hola.py

// C++ – hola.cpp (compilación única)
#include <iostream>
int main() {
    std::cout << "¡Hola Mundo desde C++!" << std::endl;
    return 0;
}
// g++ -std=c++17 -O2 hola.cpp -o hola && ./hola

// Java – HolaMundo.java (compilación + ejecución)
public class HolaMundo {
    public static void main(String[] args) {
        System.out.println("¡Hola Mundo desde Java!");
    }
}
// javac HolaMundo.java && java HolaMundo
```

Ejercicio 2: Implementar Factorial

Problema: Calcular factorial(n) de forma iterativa.

- Mide el tiempo para $n = 1.000$

Python: sin límites de overflow

```
resultado = 1
for i in range(2, n + 1):
    resultado *= i # BigInt automático
```

C++: overflow en $n > 20$ con unsigned long long

```
unsigned long resultado = 1;
for (int i = 2; i <= n; i++) {
    resultado *= i; // ¡Desborda en n=21!
}
```

#Java

```
public static long factorial(int n){
    long resultado = 1;
    for (int i = 2; i <= n; i++) resultado *= i;
    return resultado;
}
```

Midiendo los tiempos

Python	<pre>import time ... start = time.time() //Código a medir end = time.time() print("Tiempo: {(end - start):.4f} segundos")</pre>
C++	<pre>#include <chrono> ... auto start = high_resolution_clock::now(); //Código a medir auto end = high_resolution_clock::now(); auto duration = duration_cast<milliseconds>(end - start); cout << "Tiempo: " << duration.count() << " ms" << endl;</pre>
Java	<pre>long start = System.nanoTime(); //Código a medir long end = System.nanoTime(); Double duration = (endTime - startTime) / 1000000.0; System.out.println("Tiempo: " + duration + " ms");</pre>

Algunas Diferencias

Sintaxis	Python	C++	Java
Declaración	<code>x = 5</code>	<code>int x = 5;</code>	<code>int x = 5;</code>
Array / Lista	<code>arr = [1,2,3]</code>	<code>int arr[] = {1,2,3};</code>	<code>int[] arr = {1,2,3};</code>
Tamaño	<code>len(arr)</code>	<code>sizeof(arr)/sizeof(arr[0])</code>	<code>arr.length</code>
Función	<code>def f(x):</code>	<code>int f(int x) {}</code>	<code>int f(int x) {}</code>
Error índice	<code>IndexError</code>	<code>std::out_of_range</code>	<code>ArrayIndexOutOfBoundsException</code>

Gestión de ejecución	Python	C++	Java
Memoria	GC automático	Manual / RAI	GC automático
Compilación	Interpretado	Compilado nativo	Bytecode + JIT

Ejercicio 3: Sumando matrices

Python	<pre>import numpy as np N = 10000 def suma_numpy(): matrix = np.zeros((N, N), dtype=np.int32) for i in range(N): for j in range(N): matrix[i][j] = (i + j) % 100 total_sum = np.sum(matrix) print(f"Suma total: {total_sum}")</pre>	<pre>N = 10000 def suma_pure_python(): matrix = [[0] * N for _ in range(N)] for i in range(N): for j in range(N): matrix[i][j] = (i + j) % 100 total_sum = 0 for i in range(N): for j in range(N): total_sum += matrix[i][j] print(f"Suma total: {total_sum}")</pre>
C++	<pre>#include <chrono> ... auto start = high_resolution_clock::now(); //Código a medir auto end = high_resolution_clock::now(); auto duration = duration_cast<milliseconds>(end - start); cout << "Tiempo: " << duration.count() << " ms" << endl;</pre>	

Ejercicio 3: Sumando matrices

Java

```
long start = System.nanoTime();  
//Código a medir  
long end = System.nanoTime();  
Double duration = (endTime - startTime) /  
1000000.0;  
System.out.println("Tiempo: " + duration +  
" ms");
```

Ejercicio 3: Sumando elementos de matriz

- Vamos a hacer un método que sume todos los elementos de una matriz bidimensional.
- Vamos a los tiempos de ejecución en cada lenguaje (Python, C++ y Java) considerando matrices enteras de gran escala y usando arrays.

Array `int[N][N]` donde $N = 10000$ y 20000

Tabla Comparativa de Rendimiento

Tamaño	Python	C++	Java
N = 10000	ms	ms	ms
N = 20000	ms	ms	ms

Juez Virtual

- Plataforma automática que evalúa el código que envías ejecutándolo contra un conjunto de pruebas.
- Entrada/Salida Estándar: Tu programa debe leer de la entrada estándar (stdin / cin / Scanner) y escribir en la salida estándar (stdout / cout / System.out).
- Veredicto automático:
 - **AC** (Accepted): ¡Solución correcta!
 - **WA** (Wrong Answer): La salida no coincide con la esperada (¡ojo con espacios, saltos de línea y formatos!)
 - **TLE/MLE** (Time/Memory Limit Exceed): Tu programa es demasiado lento o consume demasiada memoria.
 - **RTE** (Runtime Error)/**CE** (Compilation Error): Error en ejecución o compilación.

Ejercicio 4: Usando el Juez Virtual

- En laboratorio vamos a usar DOMJudge

URL: <https://complicaus.eii.us.es>

- Crear un programa que use el método factorial, tomando varios números enteros de la entrada estándar y devuelva el factorial de cada uno (por la salida estándar).
- Caso de prueba:

Entrada (stdin)	Salida (stdout)
10	3628800
50	815915283247897734345611269596115894272000000000

Cierre y actividad para casa

Benchmark de Rendimiento en Ordenación.

Escribe un programa en Python, C++ y Java que realice las siguientes acciones:

1. Generar una lista/vector de 10.000.000 de números enteros aleatorios.
2. Medir el tiempo exacto que tarda en ordenarlos utilizando las funciones nativas de cada lenguaje:
 - Python: `list.sort()` (Timsort)
 - C++: `std::sort()`
 - Java: `Arrays.sort()`
3. (Opcional en Python): Comparar el tiempo de `list.sort()` frente a una implementación propia del algoritmo de la Burbuja con solo 20.000 elementos.

Plataformas de programación competitiva

Plataforma	Recomendación	Por qué
Codeforces	C++17	STL, velocidad, estándar de facto
AtCoder	C++17	Mismo ecosistema, librerías estándar
LeetCode	C++ o Python	Flexibilidad según el problema
ICPC	C++17	Líder indiscutible en competición
Kaggle / Data	Python	Ecosistema científico completo

Recursos y Referencias

Recurso	Enlace	Para qué
Python Tutorial	docs.python.org/3/tutorial/	Sintaxis y buenas prácticas
Oracle Java	docs.oracle.com/javase/tutorial/	Fundamentos de Java
LearnCpp	learncpp.com	C++ moderno paso a paso
C++ Reference	cppreference.com	Documentación completa
CP Handbook	cses.fi/book/book.pdf	Biblia de programación competitiva (gratis)